LOW LEVEL DESIGN

The LLD Runbook

Ten cases, from the first sentence to the follow-up.

Part 1 closed on the sentence this half exists to answer: the interviewer changes their mind, and both of you find out what your design is made of. Every case here runs to that moment, and then keeps going.

The first two are rounds I sat in myself — the prompt as it was worded, the design I actually wrote, mistakes left in. The other eight I work from the other side of the table: problems I set as an interviewer and take apart with people I mentor. Attempt each one before you read on. The value isn't in my answer; it's in finding where your instincts and mine part company.

Varun Kashyap

Contents

How to work a case

1 Attempt
Stop at the brief and design it yourself before turning the page.

2 Compare
Check ownership, invariants and one full flow — not class names.

3 Adapt
Answer the follow-up out loud before reading how I answered it.

P1 LRU Cache

The round I misread

The invite said low level design. The interviewer joined, shared a live code editor, and read out a problem that never used the word cache once. I had about a minute to decide what kind of round this was.

> WHAT I WAS ASKED
>
> “We show a user's location and address on the checkout page, and it needs to be near real time. The user base is large, so give priority to the people who come back often. We can afford to be a little slower for everyone else.”
>
> WHAT I SAID BACK
>
> “This sounds like a cache. *Come back often* could mean lifetime visit count or recent activity — I read it as recency, so I'd start with least-recently-used eviction and a fixed capacity. Tell me if you meant frequency and I'll change the rule.”

I drew nothing. I repeated the problem back, named the shape I thought it had, and put one assumption on the table — the opening move from section 6.1, more or less by reflex. That last clause is the part worth stealing: naming the interpretation you *rejected* costs four seconds and tells the interviewer you saw the ambiguity rather than missed it.

Q1 What am I actually building?

A couple of things needed pinning down before I wrote anything. I asked three, and only one of them changed the design.

Does a read count as a use?	Yes. `get` promotes the key it finds.
Bounded by entries or by memory?	Entries. Capacity is fixed when the cache is built, and never grows.
Does anything expire on its own?	No. An entry leaves only when a new one needs its place.

The first row is the one that pays. If a read promotes the key, `get` is a mutating operation — and that single fact decides your data structure before you have drawn a box. The other two bound the problem without changing it.

We settled the rest out loud: single-threaded, exact recency rather than an approximation, integer keys and values. Expiry, persistence, cache warming and anything distributed were agreed out of scope. In a live-coding round, *say* the scope rather than typing it — the editor is for code, and a comment block of assumptions reads as stalling.

> YOUR TURN
>
> You now have exactly what I had: the prompt, three answers and the agreed scope. Give it fifteen minutes before you turn the page. Write down the classes you would keep, the public API, and one sentence on what you would allow to vary later. Not the implementation — the shape.

P1 LRU Cache

Two structures, one promise

Everything below follows from one promise: find any key in constant time, and move it to the front in constant time. Miss either half and this is just a slower map.

Q2 What are the things?

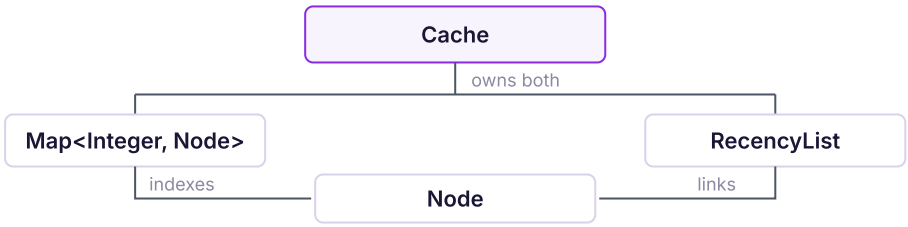
Three nouns from the prompt did not survive: `User`, `Address` and `CheckoutPage`. They are the reason the cache exists, not parts of it, and pulling them in would tie a general mechanism to one screen. Say your rejects out loud in the room — discarding a plausible noun scores better than listing it.

Q3 Who owns what?

What remained, and what each one is answerable for:

`Cache`	Owns both structures and leaves them consistent after every call. Nothing outside it touches either one.
`Map<Integer, Node>`	Finds the node for a key. Mapping to values instead would force a scan to reorder.
`RecencyList`	Holds the order. Relinks a node it is handed, and never searches for one.
`Node`	Key, value, both links. It carries its own key so eviction can clean the index.

Two invariants hold the whole thing up: **every key has exactly one node, present in both structures**, and **size never exceeds capacity**. Name them while you design. Every bug in this problem is one of the two breaking.



The design as I wrote it. `Cache` decides which node leaves.

I wrote the signatures before any of the bodies. That is worth doing even when you are typing rather than drawing, because it forces the ownership above to be a decision rather than an accident.

THE PUBLIC SURFACE

```
public class Cache {
    private final Map<Integer, Node> index;
    private final RecencyList order;
    private final int capacity;

    public Integer get(int key)         { /* promotes on a hit */ }
    public void put(int key, int value) { /* evicts when full  */ }
}
```

Q4 What will change?

He asked what I thought would change. I said nothing — I had been asked for an LRU cache, so `put` takes the tail and there is nothing to vary. I used the time I had left to make the code clean.

P1 LRU Cache

What six calls proved

The code was done well before time. He wanted to see it run, so we walked inputs through it together in the editor rather than reading the implementation back.

Q5 Does it actually work?

Capacity three, starting empty:

CALL	RETURNS	ORDER, MOST RECENT FIRST	WHAT HAPPENED
`put(1, 10)`	—	`1`	Added to both structures.
`put(2, 20)`	—	`2, 1`	New entry becomes head.
`put(3, 30)`	—	`3, 2, 1`	Full. Nothing evicted yet.
`get(1)`	`10`	`1, 3, 2`	A read reorders.
`put(4, 40)`	—	`4, 1, 3`	Tail `2` leaves both.
`put(1, 90)`	—	`1, 4, 3`	Update in place.

Rows four and six are the ones to rehearse. The fourth proves a read is a write, which is the clarification from the brief finally paying out. The sixth is the case candidates drop: overwriting a key that already exists must not create a node and must not evict anything. If your `put` inserts blindly, that is where it breaks, and it is usually the first thing an interviewer tests.

> THE BRANCH ROW SIX EXERCISES

```
public void put(int key, int value) {
    Node hit = index.get(key);
    if (hit ≠ null) {
        hit.value = value;
        order.moveToFront(hit);
        return;                  // no new node, nothing evicted
    }
    if (index.size() == capacity) evictOne();
    index.put(key, order.addFront(key, value));
}
```

Derive the cost here, at the call site, rather than announcing Big-O at the end. Every line above is one hashed lookup or a fixed number of pointer writes, so `get` and `put` are both **O(1) average**, in **O(capacity)** space for nodes held by two structures at once. The worst case belongs to the hash function, not to this design — worth saying, because it shows you know which part you control.

The two bugs that show up here are both invariant breaks: a `get` that returns the value without reordering, which passes every test that ignores eviction order; and eviction that cleans one structure and forgets the other, which is exactly why `Node` carries its own key. An access-ordered `LinkedHashMap` avoids both in about five lines — the right answer in production, the wrong one here, because it hides the mechanism you were asked to demonstrate.

P1 LRU Cache

The question after the code worked

We finished the trace and the code was correct. That was the moment I relaxed, and the moment the round actually began, with about thirty minutes still on the clock.

> THE FOLLOW-UP
>
> "What happens to your code if tomorrow product wants it faster for the people who use it *least*, rather than the most recent ones?"

I panicked, then took two silent minutes — easily the best decision of that round. Section 6.2's four moves:

1. **Restated:** same storage, same API, a different entry chosen for eviction.
2. **Located:** one line inside `evictOne`, the one that takes the tail.
3. **Separated:** index, list and both O(1) guarantees untouched; only the victim choice moves.
4. **Admitted:** the rule was hard-coded, so I named where it should have been swappable.

Making that one choice swappable is exactly what **Strategy** does — section 5.6's signal is one decision that keeps changing inside a settled object. Not State: the cache is handed its policy.

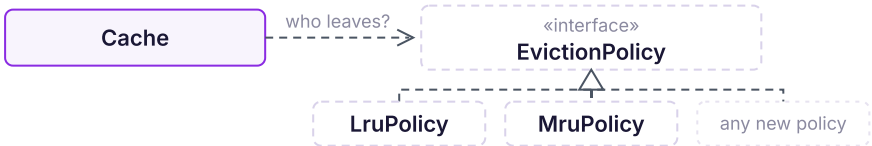
BEFORE — THE RULE IS A LINE OF CODE

```
private void evictOne() {
    Node victim = order.last();              // "least recently used"
    order.remove(victim);
    index.remove(victim.key);
}
```

AFTER — THE RULE IS A COLLABORATOR

```
private void evictOne() {
    Node victim = policy.select(order);      // tail for LRU, head for MRU
    order.remove(victim);
    index.remove(victim.key);
}
```

He asked what a third rule would cost. One new class, and four or five lines of wiring.



Dashed boxes arrived with the follow-up. A third rule is one more box on the same bar.

Say the limit out loud too: LFU fits this interface but not this structure, because it needs frequency counts the recency list never keeps. `github.com/varunkashyap-vkd/lld-runbook/lru-cache`

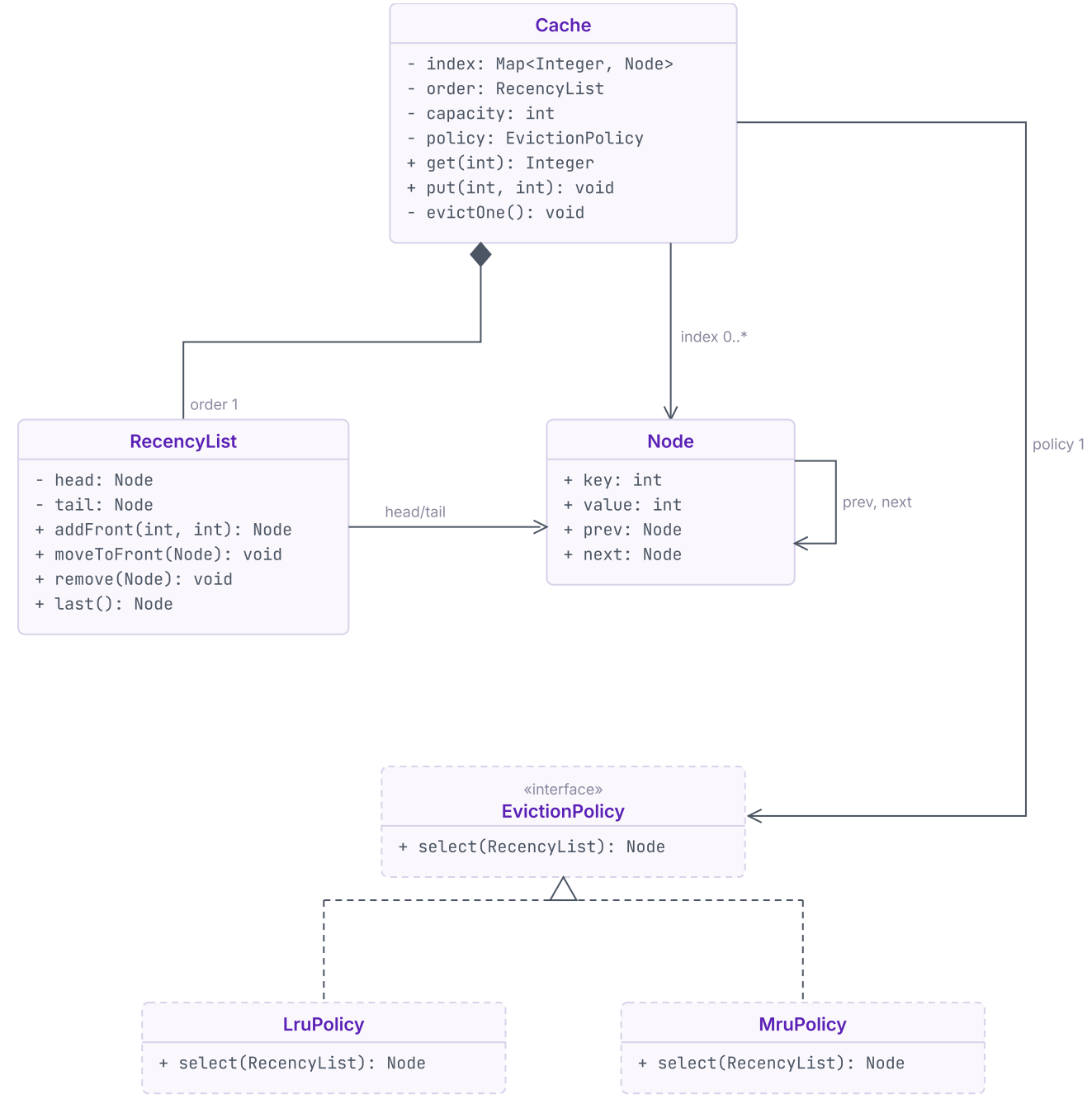
What he was probing: not whether I could write an LRU cache, but whether it would survive a product decision.

Practise next: make it thread-safe without breaking either invariant, then design LFU and name what has to change.

If you take one thing from this problem: a correct first version is evidence, not the finish line.

P1 LRU Cache

Every class, and how they connect



composition · association · implements

added later · + public · - private

Dashed classes arrived with the follow-up. Full implementation: `github.com/varunkashyap-vkd/lld-runbook/lru-cache`

P2 Google Docs

Ten minutes deciding what not to build

An object-oriented design round in an SDE-III loop at a big tech company. I had met this problem before, but only ever as high level design. This one asked what was inside the boxes.

WHAT I WAS ASKED

"Everyone here uses Google Docs for design discussions and we want to drop the dependency. Say we're building the internal alternative. The high level design is done and approved — the low level implementation is yours."

WHAT I SAID BACK

"That's too big to finish in one sitting, so let's start with the basics. Three things I think this round is about: everyone with the document open sees the same content, several people can type at once, and an editor can undo their own changes. If that's the wrong three, redirect me now."

Notice what that is not. It is not *what would you like me to focus on?* An unbounded prompt is an invitation to bound it, and handing the job back reads as avoidance. Propose a scope and invite correction.

Q1 What am I actually building?

He took the three. Three more things needed pinning down before any of it became classes.

Where does the document live?	Behind a repository. A server owns storage and hands back a document by id.
Whose change does undo reverse?	The editor's own last change, not the document's.
Who is allowed to edit?	One owner, who shares with viewers and editors. Permission is checked before an edit is accepted, not after.

The middle row is the one that pays. *Undo* sounds like one word and is two designs: reversing the document's last change is a shared stack anyone can pop, while reversing your own is a stack per person. He wanted the second, so undo history cannot live on the document — a decision that costs nothing at minute eight and a rewrite at minute thirty.

The first row I answered myself instead of asking. I said the database is not this round, drew the line, stated what I would assume past it, and moved on. When the high level design is already approved, naming what you are **not** designing is part of the answer, not a way of dodging it.

The rest went quickly. A document is a sequence of lines, and for this round every line is text. Formatting, comments, sharing links, offline editing and version history were out. He liked the narrowing — a design you can finish beats a tour of one you cannot.

YOUR TURN

You have what I had: three agreed features, three answers and a boundary. Give it fifteen minutes. Write the classes you would keep, the public API, and what happens when two people edit the same line at once.

P2 Google Docs

An edit has to carry its own past

Everything follows from one decision: the unit of change is not a keystroke but an object that knows which line it touched, what was there before, and what replaces it.

Q2 What are the things?

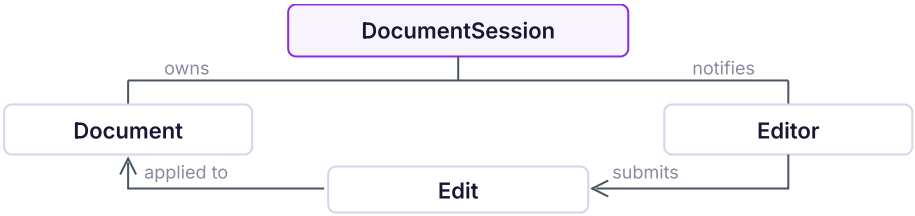
Four nouns did not survive. `Page` is a renderer's decision and it moves when the zoom does. `Cursor` belongs to the client. `Character` sits below the unit we had just agreed on. `Database` is past the boundary. Say the rejects out loud — discarding a plausible noun scores better than listing it.

Q3 Who owns what?

Four survived, each answerable for something the others must not touch.

`Document`	Owns the lines and when each was last accepted. The only thing that mutates content.
`Edit`	One change, immutable: line, text before, text after, who, when. Carrying *before* lets one object be both checked and reversed.
`Editor`	One participant. Holds their role and their own undo stack; nobody else's is on it.
`DocumentSession`	Holds the document and everyone in it. Checks permission, applies, broadcasts.

Two invariants: **content changes only through an accepted `Edit`**, so no path writes a line directly; and **an editor's undo stack holds only their own accepted edits**. Break the second and undo restores text nobody saw.



`Document` holds a `List<String>`, and the session is the only thing that talks to both sides.

I wrote the signatures before any of the bodies. The record is the interesting half.

THE PUBLIC SURFACE

```
public record Edit(int line, String before, String after, UserId by, Instant at) { }

public final class DocumentSession {
    private final Document document;
    private final Map<UserId, Editor> present;

    public boolean submit(Edit edit) { // permit, apply, broadcast }
    public boolean undo(UserId who)  { // their last edit, reversed }
}
```

Q4 What will change?

I said the conflict rule. Last write wins is the simplest thing that could work and I expected product to argue with it first, so the comparison sits in one method I could replace. What a line *is* I did not treat as variable. The round said text.

P2 Google Docs

Two editors, one line

A model like this is worth exactly what it does when two people land on the same line before either has seen the other. Everything else is bookkeeping around that moment.

Q5 Does it actually work?

Line 4 reads `Latency budget: 200ms`. Ana and Ben both have the document open.

WHAT ARRIVES	LINE 4	WHAT HAPPENED
Ana, `200ms` to `150ms`, 10:04:01	`150ms`	*before* matches what is there. Accepted, pushed to Ana's stack, broadcast.
Ben, `200ms` to `300ms`, 10:04:03	`300ms`	*before* is stale — Ben never saw Ana's line. Later timestamp, so it wins.
Ben's edit reaches Ana	`300ms`	Her screen corrects itself. Her undo stack still holds her own edit.
Ana presses undo, 10:04:06	`200ms`	Sent as a fresh edit, *before* `300ms`. Later again, so it is accepted.

Row four is the one to sit with. Ana's undo is correct by the rule we agreed and wrong by any human standard — she has just deleted Ben's change without being told he made one. Last write wins cannot tell an undo from an edit. Say that out loud when you reach it: naming the case your rule handles badly beats a fifth row pretending it does not exist.

INSIDE DOCUMENT — THE DECISIVE CHECK

```
boolean apply(Edit e) {
    if (e.line() < 0 || e.line() ⩾ lines.size()) return false;
    if (lines.get(e.line()).equals(e.before())) return accept(e);  // nobody moved
    return e.at().isAfter(lastAcceptedAt.get(e.line())) && accept(e);
}
```

Undo needs no machinery of its own. It is not a rewind — it is a new edit that points backwards, so it travels the same path and obeys the same rule as everything else. That is the return on modelling a change as an object: the second mechanism cost one method.

THE SECOND MECHANISM, FOR FREE

```
public boolean undo(UserId who) {
    Edit mine = present.get(who).undoStack().poll();
    if (mine == null) return false;
    String now = document.lineAt(mine.line());
    return submit(new Edit(mine.line(), now, mine.before(), who, Instant.now()));
}
```

Derive the cost here rather than announcing it at the end. `apply` is a bounds check, one comparison and one write — **O(1)** in the number of lines, linear only in the length of the line it compares. The broadcast is **O(n)** in editors present, and that *n* is what this design actually scales by.

Three cases to have an answer ready for: an edit whose line no longer exists, because a deletion arrived first; two edits carrying the same timestamp, which needs a tiebreak — editor id will do — or the winner depends on map ordering, which is not a design; and an undo stack that grows all session, which is a memory leak with a friendly name.

P2 Google Docs

The assumption inside the word “text”

The model had answers for the awkward rows and the round felt finished. It was not. He reached past all of it and picked up the one simplification I had proposed myself.

> THE FOLLOW-UP
>
> “How do your document and your conflict resolution hold up when a line isn't text? Images, diagrams — where do those fit?”

Having your own scope reduction handed back is a particular kind of exposed. I had said *text only*, he had agreed, and half an hour later it was mine to answer for. Section 6.2's four moves:

1. **Restated:** a position stops being a string, and merging depends on what sits at it.
2. **Located:** the list type in `Document`, and the comparison inside `apply`.
3. **Separated:** session, permissions, broadcast and undo untouched; the element changes.
4. **Admitted:** I made the conflict rule replaceable and the content type fixed. Wrong one.

BEFORE — THE DOCUMENT KNOWS WHAT A LINE IS

```
private final List<String> lines;

boolean apply(Edit e) {
    if (lines.get(e.line()).equals(e.before())) return accept(e);
    return e.at().isAfter(lastAcceptedAt.get(e.line())) && accept(e);
}
```

AFTER — THE POSITION DECIDES

```
private final List<DocElement> elements;

boolean apply(Edit e) {
    return elements.get(e.line()).merge(e);    // text, image, diagram
}
```

The old rule is not deleted, it is moved: `TextBlock`, the first `DocElement`, merges exactly as above. The second implementation is what the question was for. `ImageBlock` refuses to merge at all — an image is replaced whole, so two people replacing it is a conflict you surface rather than resolve. Diagrams later are one more class and nothing else. That is Open/Closed with the axis finally chosen correctly, and it needs no pattern name to be worth marks.

He asked what the change would cost, and it does not stop at `Document`. `Edit` declares *before* and *after* as `String`, and that record sits on every undo stack, inside every broadcast and on the wire. Make the position polymorphic without making the edit polymorphic and you get halfway: images can be held, but not described. The hard-coding was in the record, not the container — and even fixed, this model still has no word for an edit that spans positions. `github.com/varunkashyap-vkd/lld-runbook/collab-doc`

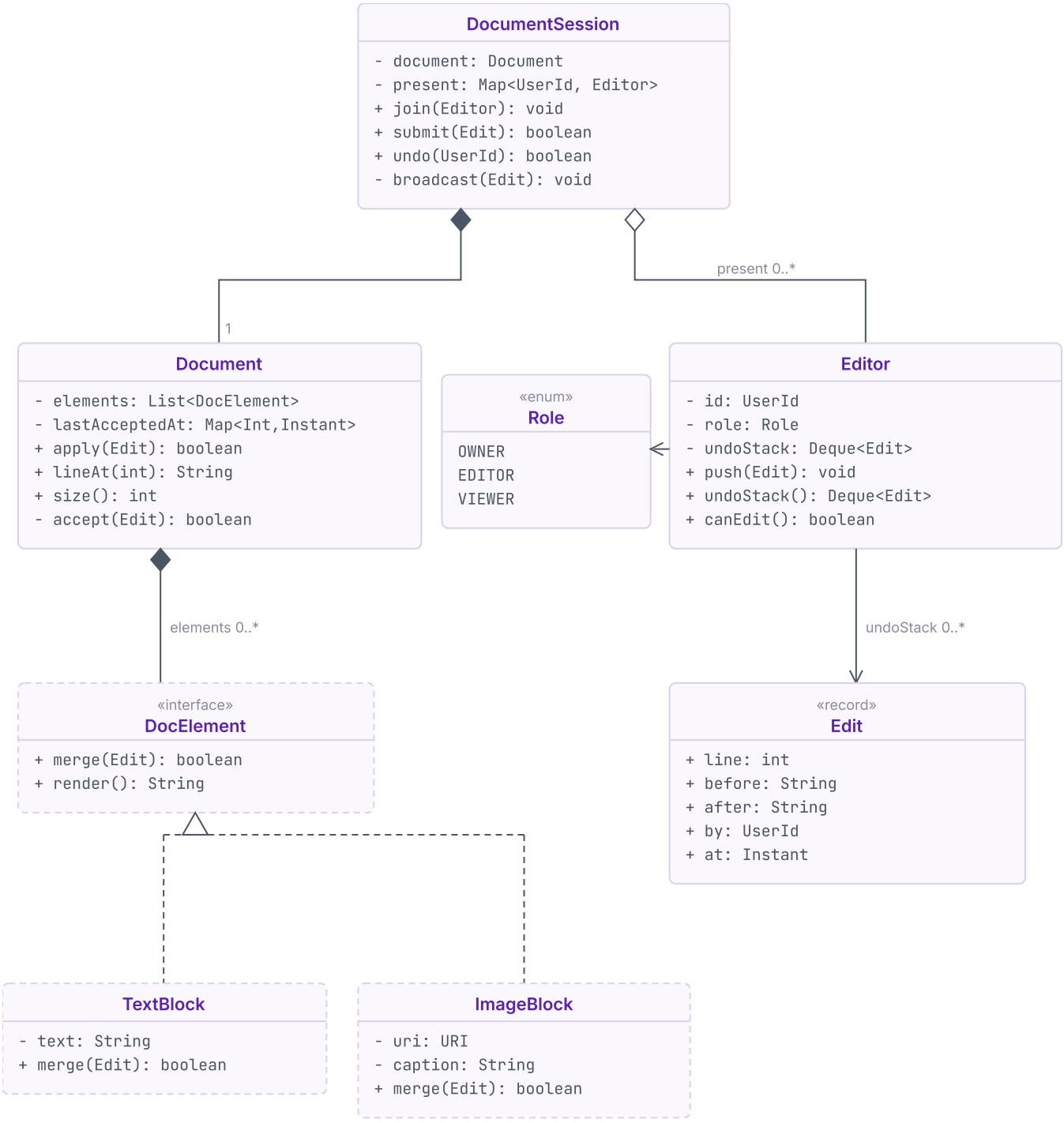
What he was probing: whether someone handed an approved high level design can produce classes that survive a change in what the product carries.

Practise next: add comments anchored to a position, then decide what an anchor means once the position it points at is deleted.

The takeaway: a simplification you propose is still your assumption. When it comes back, it is yours to answer for.

P2 Google Docs

Every class, and how they connect



composition · aggregation · association · implements

added later · + public · - private

Dashed classes arrived with the follow-up. Full implementation: `github.com/varunkashyap-vkd/lld-runbook/collab-doc`

P3 Parking lot

Everyone starts by naming the slots

This is not a round I sat. It is one I set, and one I have taken apart with enough people to know exactly where it turns. The prompt looks like a modelling exercise, and it is — just not the modelling most candidates spend their time on.

> THE PROMPT, AS I PUT IT
>
> "Design the low-level structure for a multi-level parking lot. Vehicles arrive, park and leave. The lot decides where each one goes, and has to be able to find it again."

Then I stop talking, and what happens in the next ninety seconds decides most of the round. Nearly everyone writes the same two lines first: an enum of slot sizes, and a table mapping each kind of vehicle to the size it needs. That is not wrong. It is a *decision*, and it is the one I am going to examine — but almost nobody notices they made it.

The stronger candidates stop at two sizes. The rest keep going — compact, standard, oversized, electric, accessible, motorbike — as though naming more kinds of space were the same work as designing one. Two sizes prove the point that spaces differ. Everything after that is time you are not spending on the part I am going to push on.

Q1 What am I actually building?

Three questions are worth the time here. Only one of them changes the design.

Is the layout fixed once built?	Yes. Spaces do not appear or vanish while the lot is running.
Who picks the space?	The lot does. The driver is handed a place, so allocation is a decision this design owns.
What must be true when nothing fits?	Refuse cleanly and say why. Never hand out a space a vehicle does not fit into.

The second row is the one that pays, and it is worth asking out loud even when you think you know the answer. If the driver chooses, this is a booking system and the interesting code is in the UI. If the lot chooses, you have just been handed the actual problem: an allocation rule you have to defend.

We settle the rest quickly. Several levels, a fixed layout, one entrance. Payment, pricing, reservations, number-plate recognition and anything to do with barriers are out — they are real, and none of them is what the next forty minutes are for.

> YOUR TURN
>
> Fifteen minutes, and you have what my candidates have. Write the classes you would keep and the signature of the method that parks a vehicle. Then answer one question in a sentence: what decides whether a given vehicle can go in a given space?

P3 Parking lot

A slot for every kind of vehicle

Here is the design I am handed most often, written the way a strong candidate writes it. It keeps one promise: every vehicle gets a space it fits in, and the ticket finds it again.

Q2 What are the things?

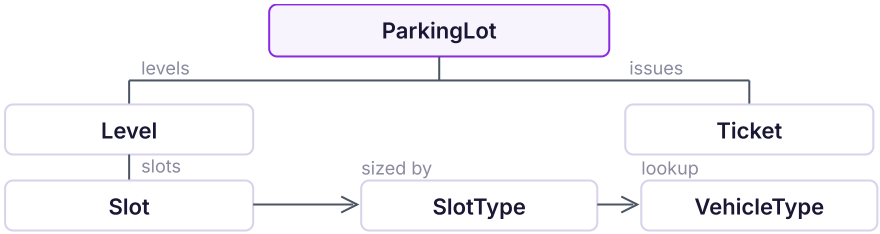
`Driver`, `Gate` and `Receipt` come up and go again — not modelled, hardware, and the payment round we scoped out. What survives is `ParkingLot`, `Level`, `Slot`, `Vehicle` and `Ticket`, plus two enums: `SlotType` and `VehicleType`.

Q3 Who owns what?

Each one is answerable for something the others must not reach into.

`ParkingLot`	Owns the levels and the allocation rule. The only thing that decides where a vehicle goes.
`Level`	Holds its slots and answers one question: is there a free slot of this size?
`Slot`	A space with a size and, when taken, an occupant. It never chooses who parks in it.
`Ticket`	Where a vehicle went and when. Issued on entry; the only way back to the slot.

Two invariants: **a slot holds at most one vehicle**, and **every parked vehicle has exactly one live ticket**. Every double-allocation bug in this problem is the first one breaking; every lost car is the second.



The baseline. A slot knows its size, and a table decides which size each vehicle needs.

The whole design turns on one line, so it is worth putting it up before anything else.

THE DECISION, IN ONE LOOKUP

```
private static final Map<VehicleType, SlotType> FITS = Map.of(
    SCOOTER, SMALL,  CAR, MEDIUM,  BUS, LARGE);

public Ticket park(Vehicle v) {
    SlotType needed = FITS.get(v.type());
    Slot free = findFree(needed);               // across every level
    if (free == null) throw new LotFullException(v.type());
    return issue(free, v);
}
```

Q4 What will change?

Most candidates say the vehicle types — a van, a lorry, a motorbike with a sidecar — so they make the enums easy to extend and move on, satisfied. Parking costs **O(levels × slots)** at worst, constant if each level indexes its free slots by size.

P3 Parking lot

A free bay, and nothing that fits it

The design above works, and candidates are usually pleased with it by this point. So I stop asking about vehicles and describe a Tuesday morning instead.

> WHAT I ASK NEXT
>
> “A bus bay has been empty since six. No buses are booked today. There are eight scooters queued at the gate and the small spaces are full. What does your lot do?”

The honest answer is that it turns them away, and the good candidates say so before I have to. Section 6.2's four moves, and the third one is where this case actually lives:

1. **Restated:** space that physically fits a vehicle must be usable by it.
2. **Located:** the `FITS` lookup, and the `type` field on `Slot`.
3. **Separated:** tickets, levels and both invariants stand; the sizing goes.
4. **Admitted:** I classified space instead of measuring it. Two enums, one too many.

BEFORE — A SPACE IS A CATEGORY

```
class Slot { int id; SlotType type; Vehicle occupant; }

SlotType needed = FITS.get(v.type());
Slot free = level.findFree(needed);        // LARGE fits buses, and only buses
```

AFTER — A SPACE IS AN AMOUNT

```
class Slot { int id; Vehicle occupant; }   // one unit of space, nothing more

int needed = v.units();                   // scooter 1, car 4, bus 8
Run free = row.findRun(needed);            // n consecutive empty slots
```

A vehicle no longer asks for a *kind* of space, it asks for an *amount*. Eight scooters take the bus bay one unit at a time, and the enum that caused the problem is deleted rather than extended. That is the move worth stealing: when a follow-up exposes a classification, the fix is usually to stop classifying, not to add a class.

It is not free, and this is the part candidates miss. Contiguity is a relationship the old model never had — an unordered `List<Slot>` cannot answer “are these eight next to each other?”, so a `Row` has to appear between `Level` and `Slot` purely to give slots neighbours. Say that cost out loud; it is the difference between having an idea and having a design. `github.com/varunkashyap-vkd/lld-runbook/parking-lot`

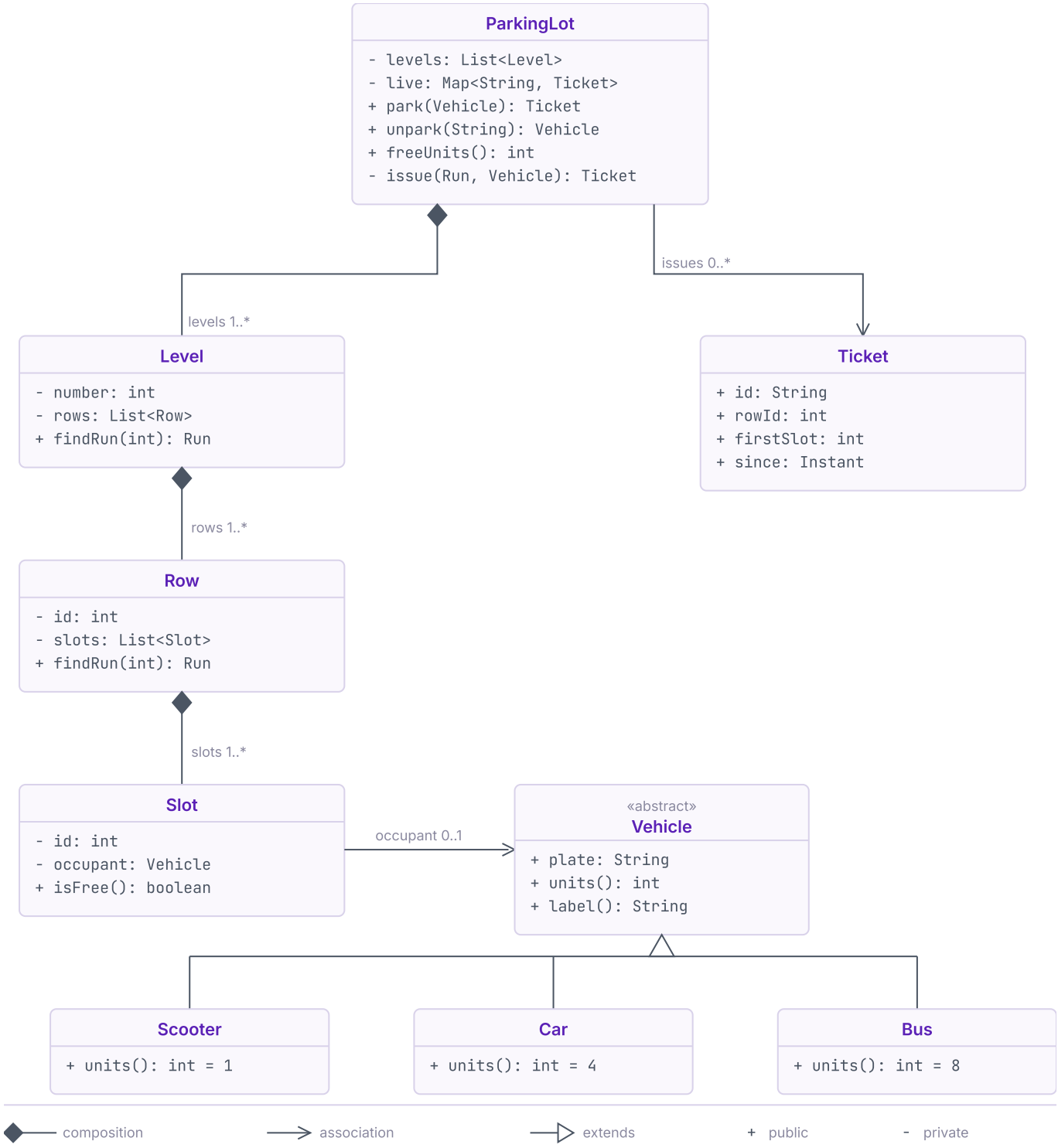
Where it stops: contiguity brings fragmentation. Eight scooters scattered one per row leave sixty free units and nowhere to put a bus. Naming that beats pretending the fix is clean.

What I am listening for: not the enum. Whether you can be shown an unused resource and reason about why your own model refused it.

The takeaway: every type you introduce is a promise that the difference matters. Two proved it; three would have been decoration.

P3 Parking lot

Every class, and how they connect



No slot type: a slot is one unit of space. Full implementation: `github.com/varunkashyap-vkd/lld-runbook/parking-lot`

P4 Vending machine

The half of the machine nobody designs

Another one I set rather than sat. It is the friendliest prompt in this book. Everyone has used a vending machine and nobody thinks it is hard. That is exactly why I watch so many confident starts go wrong here. The machine is really two machines in one box, and almost everybody designs the same one.

> THE PROMPT, AS I PUT IT
>
> "Design the low-level structure for a vending machine. It is loaded with products. It takes money from a customer and hands back the item and any change."

What comes next is remarkably consistent. Rows, a tray, an `Item` with a name and a price, a `dispense()` that turns a motor. Then money shows up as an `int`. It gets subtracted inside a method that is really about the motor, and the design moves on. The vending is finished. The payment process was never accounted for.

I once watched eleven minutes go on whether an item should carry a weight, a height and a colour. It is a vending machine. Nothing in it is going to weigh a hundred and fifty kilos, and nothing downstream reads any of the three. In that same round nobody had asked what happens when the machine runs out of ten-rupee notes.

Q1 What am I actually building?

Three questions are worth the time here. The first one is the whole case.

Can it always make change?	No. It holds a fixed stock of notes and coins. It can run out by mid-afternoon.
Where does the price live?	On the shelf, not the item. A shelf is loaded with one product at one price.
Who reloads it, and when?	An operator, between sales. Refills and price changes never land mid-transaction.

Ask the first one out loud even though you can guess the answer. The answer decides what kind of problem you have. If the machine can always make change, money is arithmetic and you have been handed a dispenser. If it cannot, every sale carries a second question. *Can I finish this?* Where you choose to ask it is the design.

I give the second one away rather than watch five minutes burn on it. Pricing an item at runtime needs an input the machine does not have. Fix the price to the shelf and load the shelf with one product. A whole category of imagined problem stops existing. The rest goes quickly. One customer at a time, so there is no concurrency to design around. Cash only, in five denominations. Out go telemetry, remote monitoring, the motor, and anything about who the customer is.

> YOUR TURN
>
> Fifteen minutes. Write the states an order passes through between the button and the tray. Write the signature of the method that takes money. Then answer one question in a sentence: what has to be true before the machine commits to a sale?

P4 Vending machine

Nothing moves until the whole sale can finish

Here is the version the strongest candidates hand me, written the way they write it. It keeps one promise. The machine never takes money for a sale it cannot complete.

Q2 What are the things?

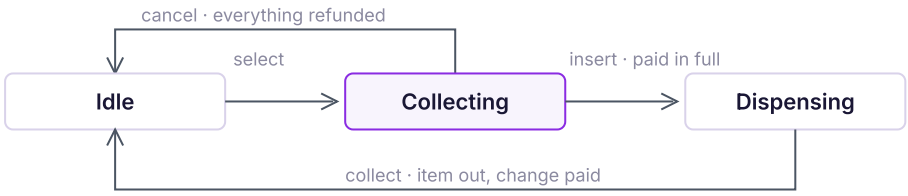
`Tray`, `Motor` and `Display` go again as hardware. `Product` goes with them. It was only a name and a price, and the shelf already owns both. What survives: `VendingMachine`, `Shelf`, `CashBox`, `Sale`, and three states.

Q3 Who owns what?

Each is answerable for something the others must not reach into.

`VendingMachine`	Owns the shelves, the cash box and the sale in flight. Delegates every transition.
`Shelf`	One product, one price, a count. Releases an item and takes a refill.
`CashBox`	The notes it holds, by denomination. Answers whether an amount can be returned.
`Sale`	The shelf chosen and the price frozen at selection. Born at the button.

Two invariants hold it together. **Money is never accepted for a sale the machine cannot complete.** And **a shelf's count drops exactly once per sale.** The first is why `canReturn` is a question, not a calculation. It is also why the lifecycle is the design here.



A short payment is not a fourth state. It leaves the machine in `Collecting`.

Everything the case turns on lives inside one transition. It goes up before anything else.

> COLLECTING, DECIDING WHETHER TO COMMIT

```
SaleState insert(VendingMachine m, int note) {
    m.cash().accept(note);                // taken and kept, one step
    m.sale().add(note);
    if (m.sale().shortfall() > 0) return this;
    if (!m.cash().canReturn(m.sale().change())) return cancel(m);
    return new Dispensing();
}
```

Feed a twenty at a fifty-rupee shelf. `shortfall()` is thirty, the method returns `this`, and the machine stays in `Collecting`. If that needs special handling, your states are wrong.

Q4 What will change?

Almost everyone says the products and the prices. They are right, and it costs nothing. A shelf is loaded, not subclassed, and `refill` is the whole of restocking. On cash this is a good design.

P4 Vending machine

The states were named after the money

It survives everything I throw at it in notes and coins. So I stop talking about coins.

> WHAT I ASK NEXT
>
> "Product wants to take cards from next quarter. Tap and go. How much of this survives?"

The first answer is nearly always a branch. A `CARD` case inside `insert`. Fair instinct, wrong answer. The lifecycle does not have a cash branch in it. It *is* cash. So I walk it through section 6.2's four moves.

1. **Restate it:** the machine has to take a second kind of money, and a third after that.
2. **Where it lands:** not in `insert`, where you would expect. In `canReturn`, and in the name `Collecting`.
3. **What stays, what changes:** shelves, counts and invariants hold. Tracking what is paid leaves `Sale`.
4. **What I got wrong:** I named the states after cash, so they could only ever run cash.

BEFORE — THE TRANSITION KNOWS WHAT MONEY IS

```
SaleState insert(VendingMachine m, int note) {
    m.cash().accept(note);                 // taken and kept, one step
    m.sale().add(note);
    if (m.sale().shortfall() > 0) return this;
    if (!m.cash().canReturn(m.sale().change())) return cancel(m);
    return new Dispensing();
}
```

AFTER — THE TRANSITION ASKS, THE INSTRUMENT ANSWERS

```
SaleState pay(VendingMachine m, PaymentMethod p) {
    if (!p.hold(m.sale().due())) return this;   // held, not kept
    return new Dispensing();
}
SaleState collect(VendingMachine m) {           // in Dispensing
    m.sale().shelf().releaseOne();
    m.payment().capture();                      // kept only now
    return new Idle();
}
```

One question replaces three, and all three that left were cash nouns. But look at where `capture()` went. Cash leaves no gap between taking money and keeping it. A card authorises first and captures after, so `CashPayment` must **hold** notes it can still hand back. `github.com/varunkashyap-vkd/lld-runbook/vending-machine`

That gap is what makes the sale **atomic**. Money is held across the dispense, not kept before it, so a jam is recoverable: release the payment and the customer gets their money back. The invariant said the machine never takes money for a sale it cannot complete. Read it again. It was only ever about the change.

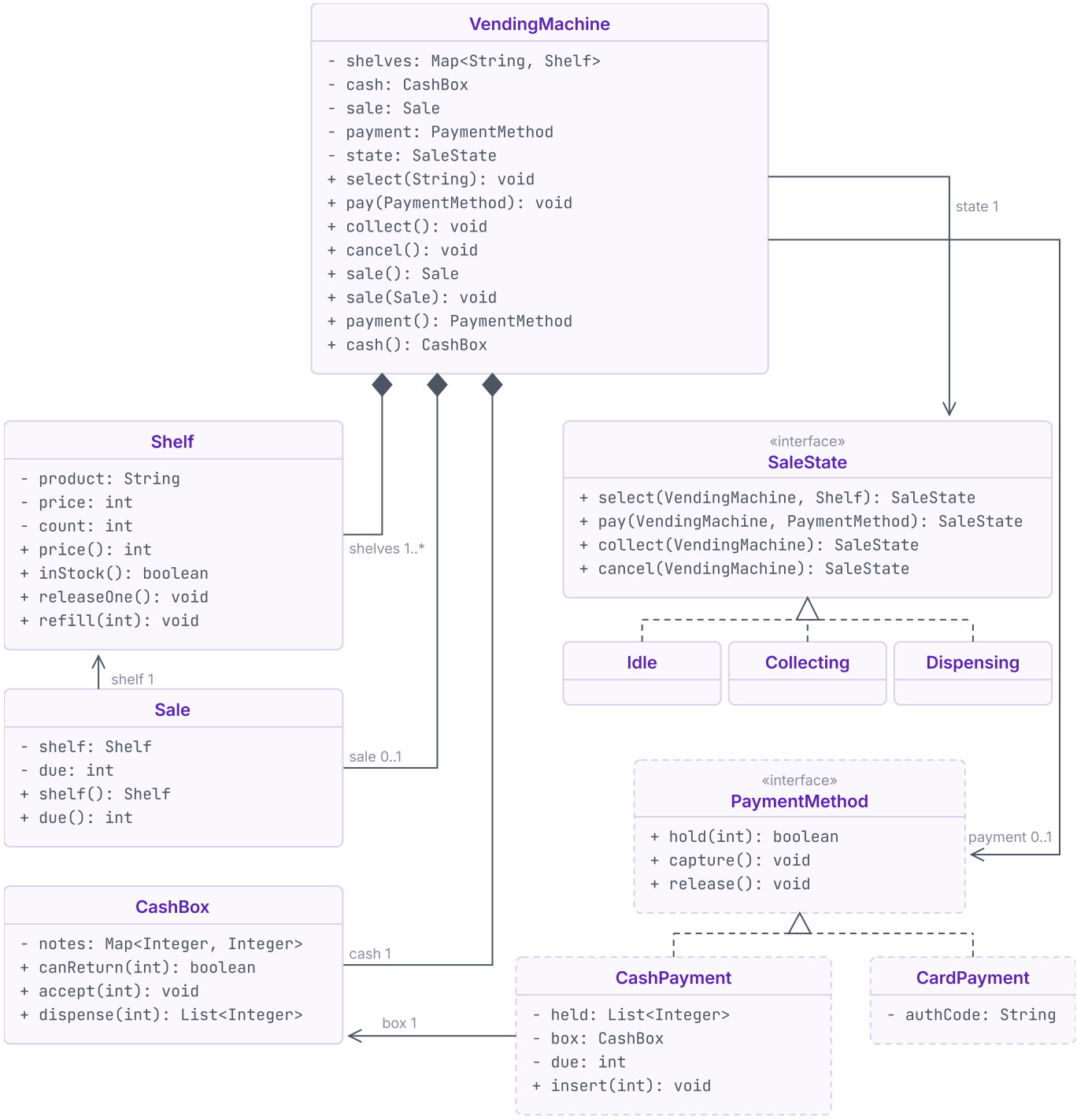
Where it stops: atomicity ends at the dispense. A card release can be refused an hour later, so the machine can reach `Idle` still owing money.

What I am listening for: whether you notice one of your states is named after a coin, and what you say when the motor jams. **Two to try:** the customer walks away mid-sale. The shelf sells out between select and pay.

If you take one thing from this: name a state for the decision it is waiting on, never for the thing it is waiting with.

P4 Vending machine

Every class, and how they connect



`hold`, `capture` and `release` are the atomicity contract: nothing is kept until `collect` has released the item. Full implementation: `github.com/varunkashyap-vkd/lld-runbook/vending-machine`